Kazalo vsebine

1. Povzetek tehnične specifikacije

Ta dokument vsebuje tehnično specifikacijo za implementacijo P0 prioritete RestaurantOS v1.0.0. Specifikacija je namenjena razvojni ekipi (1 FTE fullstack developer + 0.3 FTE designer) in vsebuje API contracts, TypeScript sheme, komponentne specifikacije in acceptance criteria za vsako od treh P0 komponent: FURS produkcijska certifikacija, Stripe plačilni gateway in PWA aplikacija.

Specifikacija temelji na GAP analizi (september 2025), ki je pokazala, da je 73% kode že production-ready. Ta dokument podrobno opiše preostalih 27% - torej 42 človek-dnevov dela, razdeljenih na 8 tednov. Vsaka komponenta ima definirane: (1) obstoječe module, (2) nove endpoint-e, (3) TypeScript sheme, (4) React komponente, (5) acceptance criteria in (6) testne scenarije.

POMEMBNO OBVEŠČANJE

FURS je 15. septembra 2025 rotiral signing certifikate. Približno 8000 certifikatov bo poteklo december 2025/januar 2026. RestaurantOS že ima cert-status API, ki opozarja pred potekom - to je treba aktivirati v produkciji takoj po P0-1 implementaciji.

2. P0-1: FURS produkcijska certifikacija

FURS (Finančna uprava Republike Slovenije) zahteva davčno potrjevanje računov po zakonu ZDDV-1. RestaurantOS mora imeti veljaven .p12 certifikat, ki ga FURS izda na zahtevo davčnega zavezanca. Ta specifikacija opisuje admin UI za upload certifikata in postopek validacije v testnem in produkcijskem okolju.

2.1 Obstoječi moduli (production-ready)

Modul	Datoteka	Status	Opis
PKCS12 Loader	src/lib/furs/crypto/pkcs12-loader.ts	Production	OpenSSL CLI + Node crypto fallback
ZOI Generator	src/lib/furs/crypto/zoi.ts	Production	MD5 hash za ZOI
PEM Loader	src/lib/furs/crypto/pem-loader.ts	Production	PEM format branje
Config Resolver	src/lib/furs/config-resolver.ts	Production	Per-location 4-nivojski fallback
Token Manager	src/lib/furs/api/token.ts	Production	OAuth token z auto-refresh
Build Request	src/lib/furs/api/build-request.ts	Production	SOAP zahtevek konstrukcija
Verify Invoice	src/lib/furs/api/verify-invoice.ts	Production	FURS REST API klic
Cert Status API	src/app/api/furs/cert-status/route.ts	Production	Preverjanje veljavnosti certifikata
FURS Route	src/app/api/furs/route.ts	Production	Glavni FURS endpoint

Tabela 2.1: Obstoječi FURS moduli

2.2 Nov API endpoint: POST /api/furs/cert-upload

Nov endpoint za varni upload .p12 certifikata. Datoteka se shranjuje v datotečni sistem (./certs/furs-{locationId}.p12) z .gitignore zaščito. Path se zapiše v Location.fursCertPath polje.

Request

```
POST /api/furs/cert-upload
Content-Type: multipart/form-data
Authorization: Bearer <admin-token>

Form fields:
  cert: File (.p12, max 100KB)
  password: string (cert password)
  locationId: string (optional, default = first active)
  environment: "test" | "production"
```

TypeScript shema (Zod)

```
// src/lib/validations/furs.ts
import { z } from 'zod'

export const uploadCertSchema = z.object({
  password: z.string().min(1).max(256),
  locationId: z.string().cuid().optional(),
  environment: z.enum(['test', 'production']).default('test'),
})

export type UploadCertInput = z.infer<typeof uploadCertSchema>

// Response
export interface CertUploadResponse {
  success: boolean
  certPath: string  // relativna pot za DB
  environment: 'test' | 'production'
  validUntil: string  // ISO date
  issuer: string  // FURS CA info
  message: string
}
```

Implementacija (route handler)

```
// src/app/api/furs/cert-upload/route.ts
import { NextResponse } from 'next/server'
import { writeFile, mkdir } from 'fs/promises'
import path from 'path'
import { requireAuth } from '@/lib/auth-middleware'
import { uploadCertSchema } from '@/lib/validations/furs'
import { loadFromPKCS12 } from '@/lib/furs/crypto/pkcs12-loader'
import { getCertInfo } from '@/lib/furs/crypto/certificates'
import { db } from '@/lib/db'
import { logger } from '@/lib/logger'

export const dynamic = 'force-dynamic'

export async function POST(req: Request) {
  // 1. Auth check (samo admin)
  const auth = await requireAuth(req, { permission: 'admin' })
  if (auth.error) return auth.error

  // 2. Parse multipart
  const formData = await req.formData()
  const file = formData.get('cert') as File
  const password = formData.get('password') as string
  const locationId = formData.get('locationId') as string | null
  const environment = (formData.get('environment') as 'test' | 'production') || 'test'

  if (!file || !password) {
    return NextResponse.json(
      { error: 'Manjka cert datoteka ali geslo' },
      { status: 400 }
    )
  }

  // 3. Validiraj velikost (max 100KB)
  if (file.size > 100 * 1024) {
    return NextResponse.json(
      { error: 'Certifikat prevelik (max 100KB)' },
      { status: 413 }
    )
  }

  // 4. Validiraj tip datoteke
  if (!file.name.endsWith('.p12') && !file.name.endsWith('.pfx')) {
```

```
  return NextResponse.json(
    { error: 'Datoteka mora biti .p12 ali .pfx' },
    { status: 400 }
  )
}

// 5. Preveri certifikat (load PKCS12)
const buffer = Buffer.from(await file.arrayBuffer())
const tempPath = `/tmp/furs-cert-${Date.now()}.p12`
await writeFile(tempPath, buffer)

try {
  const privateKey = loadFromPKCS12(tempPath, password)
  if (!privateKey) {
    return NextResponse.json(
      { error: 'Neveljavno geslo ali poškodovan certifikat' },
      { status: 400 }
    )
  }

  const certInfo = await getCertInfo(tempPath, password)
  if (!certInfo.valid) {
    return NextResponse.json(
      { error: 'Certifikat ni veljaven' },
      { status: 400 }
    )
  }

  // 6. Shrani v ./certs/ direktorij
  const certsDir = path.join(process.cwd(), 'certs')
  await mkdir(certsDir, { recursive: true })
  const finalFilename = `furs-${locationId || 'default'}-${environment}.p12`
  const finalPath = path.join(certsDir, finalFilename)
  await writeFile(finalPath, buffer, { mode: 0o600 }) // samo lastnik lahko bere

  // 7. Posodobi Location v bazi
  const targetLocationId = locationId || (await db.location.findFirst({
    where: { isActive: true },
    select: { id: true }
  }))?.id

  if (!targetLocationId) {
    return NextResponse.json(
      { error: 'Nobena aktivna lokacija najdena' },
      { status: 400 }
    )
  }

  await db.location.update({
    where: { id: targetLocationId },
    data: {
      fursCertPath: `./certs/${finalFilename}`,
      fursCertPassword: password, // NOTE: encryptiraj v produkciji
      fursEnvironment: environment,
    },
  })

  logger.info('FURS', `Certifikat uploadan za lokacijo ${targetLocationId}`)

  return NextResponse.json({
    success: true,
    certPath: `./certs/${finalFilename}`,
    environment,
    validUntil: certInfo.validUntil,
    issuer: certInfo.issuer,
```

```
      message: 'Certifikat uspešno naložen',
    })
  } finally {
    // Počisti temp datoteko
    try { await import('fs/promises').then(fs => fs.unlink(tempPath)) } catch {}
  }
}
```

2.3 Admin UI komponenta: FursCertUpload

React komponenta z drag-and-drop uploadom, geslom in environment selectorjem. Po uspešnem uploadu prikaže status certifikata (veljaven do, issuer, environment).

```
// src/components/admin/FursCertUpload.tsx
'use client'
import { useState, useCallback } from 'react'
import { useDropzone } from 'react-dropzone'
import { Loader2, Upload, CheckCircle, AlertCircle } from 'lucide-react'

interface CertInfo {
  success: boolean
  certPath: string
  environment: 'test' | 'production'
  validUntil: string
  issuer: string
  message: string
}

export function FursCertUpload({ locationId }: { locationId?: string }) {
  const [password, setPassword] = useState('')
  const [environment, setEnvironment] = useState<'test' | 'production'>('test')
  const [uploading, setUploading] = useState(false)
  const [result, setResult] = useState<CertInfo | null>(null)
  const [error, setError] = useState<string | null>(null)

  const onDrop = useCallback(async (acceptedFiles: File[]) => {
    const file = acceptedFiles[0]
    if (!file) return

    setUploading(true)
    setError(null)
    setResult(null)

    try {
      const formData = new FormData()
      formData.append('cert', file)
      formData.append('password', password)
      formData.append('environment', environment)
      if (locationId) formData.append('locationId', locationId)

      const res = await fetch('/api/furs/cert-upload', {
        method: 'POST',
        body: formData,
      })

      const data = await res.json()
      if (!res.ok) throw new Error(data.error || 'Upload failed')

      setResult(data)
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Neznana napaka')
    } finally {
      setUploading(false)
    }
```

```
}, [password, environment, locationId])

const { getRootProps, getInputProps, isDragActive } = useDropzone({
  onDrop,
  accept: { 'application/x-pkcs12': ['.p12', '.pfx'] },
  maxFiles: 1,
  maxSize: 100 * 1024,
  disabled: !password || uploading,
})

return (
  <div className="space-y-4">
    <div>
      <label className="block text-sm font-medium mb-1">Cert geslo</label>
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        className="w-full px-3 py-2 border rounded"
        placeholder="Vnesi geslo certifikata"
      />
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Okolje</label>
      <select
        value={environment}
        onChange={(e) => setEnvironment(e.target.value as 'test' | 'production')}
        className="w-full px-3 py-2 border rounded"
      >
        <option value="test">Test (FURS TEST okolje)</option>
        <option value="production">Produkcija (FURS PRODUKCIJA)</option>
      </select>
    </div>

    <div
      {...getRootProps()}
      className={`border-2 border-dashed rounded-lg p-8 text-center cursor-pointer
        ${isDragActive ? 'border-blue-500 bg-blue-50' : 'border-gray-300'}
        ${!password ? 'opacity-50 cursor-not-allowed' : ''}`}
    >
      <input {...getInputProps()} />
      {uploading ? (
        <Loader2 className="mx-auto h-12 w-12 animate-spin text-gray-400" />
      ) : (
        <Upload className="mx-auto h-12 w-12 text-gray-400" />
      )}
      <p className="mt-2 text-sm text-gray-600">
        {uploading ? 'Nalaganje...' : 'Povleci .p12 datoteko sem ali klikni za izbiro'}
      </p>
    </div>

    {error && (
      <div className="bg-red-50 border border-red-200 rounded p-3 flex items-center gap-2">
        <AlertCircle className="h-5 w-5 text-red-500" />
        <span className="text-sm text-red-700">{error}</span>
      </div>
    )}

    {result && (
      <div className="bg-green-50 border border-green-200 rounded p-3">
        <div className="flex items-center gap-2 mb-2">
          <CheckCircle className="h-5 w-5 text-green-500" />
          <span className="font-medium text-green-700">{result.message}</span>
        </div>
```

```
          <dl className="text-sm space-y-1">
            <div><dt className="inline font-medium">Okolje:</dt> <dd className="inline">{result.environment}</dd></div>
            <div><dt className="inline font-medium">Veljaven do:</dt> <dd className="inline">{new Date(result.validUntil).toLo
            <div><dt className="inline font-medium">Izdajatelj:</dt> <dd className="inline">{result.issuer}</dd></div>
          </dl>
        </div>
      )}
    </div>
  )
}
```

2.4 Acceptance Criteria za P0-1

#	Kriterij	Test scenarij	Status
AC-1	Admin lahko uploada .p12 datoteko	Upload validen certifikat → 200 OK	☐
AC-2	Neveljavno geslo zavrnjeno	Upload z napačnim geslom → 400	☐
AC-3	Prevelika datoteka zavrnjena	Upload >100KB → 413	☐
AC-4	Napačen tip zavrnjen	Upload .pdf → 400	☐
AC-5	Certifikat shranjen v ./certs/	Preveri ls certs/	☐
AC-6	Location.fursCertPath posodobljen	Preveri v DB	☐
AC-7	FURS test zahtevek uspešen	POST /api/furs → 200 z EOR	☐
AC-8	ZOI pravilno generiran	Preveri ZOI format (32 znakov)	☐
AC-9	Cert-status API vrne veljavnost	GET /api/furs/cert-status	☐
AC-10	Produkcijski preklop deluje	FURS_ENV=production → 200	☐

Tabela 2.2: Acceptance criteria za P0-1 FURS

3. P0-2: Stripe plačilni gateway

Stripe integracija omogoča natakarjem sprejemanje kartičnih plačil neposredno znotraj POS. Ta specifikacija opisuje nov API endpoint za ustvarjanje PaymentIntent, React komponento za vnos kartičnih podatkov (Stripe Elements) in webhook handler za asinhrona potrdila.

3.1 Obstoječi moduli (production-ready)

Modul	Datoteka	Status	Opis
StripeGateway class	src/lib/payment-gateways/providers/stripe.ts	Production	286 vrstic, authorize/capture/refund
Gateway Factory	src/lib/payment-gateways/factory.ts	Production	Factory pattern
Base Gateway	src/lib/payment-gateways/base.ts	Production	Abstract base class
Outbox Processor	src/lib/outbox/processors/stripe.ts	Production	Async processing
Health Check	src/lib/payment-gateways/providers/stripe.ts	Production	/v1/balance API
Gateway API	src/app/api/payment-gateways/route.ts	Production	Listing + health
Payments API	src/app/api/payments/route.ts	Production	CRUD plačil

Tabela 3.1: Obstoječi Stripe moduli

3.2 Nov API endpoint: POST /api/payments/stripe-intent

Ustvari Stripe PaymentIntent za kartično plačilo. PaymentIntent je Stripe koncept, ki sledi plačilu skozi celotni lifecycle (creation → confirmation → success/failure).

Request

```
POST /api/payments/stripe-intent
Content-Type: application/json
Authorization: Bearer <token>

{
  "amount": 4990,           // v centih (49.90 EUR)
  "currency": "eur",
  "orderId": "clxyz123...",
  "locationId": "clabc456...",
  "metadata": {
    "tableNumber": "5",
    "waiterName": "Janez"
  }
}
```

Response

```
{
  "clientSecret": "pi_3Pxyz..._secret_abc123...",
  "paymentIntentId": "pi_3Pxyz...",
  "amount": 4990,
  "currency": "eur",
```

```
  "status": "requires_payment_method"
}
```

Implementacija

```
// src/app/api/payments/stripe-intent/route.ts
import { NextResponse } from 'next/server'
import { z } from 'zod'
import { requireAuth } from '@/lib/auth-middleware'
import { validateRequest, handleApiError } from '@/lib/api-utils'
import { StripeGateway } from '@/lib/payment-gateways/providers/stripe'
import { db } from '@/lib/db'
import { logger } from '@/lib/logger'

const intentSchema = z.object({
  amount: z.number().int().min(50).max(99999999), // 0.50 EUR to 999.999,99 EUR
  currency: z.enum(['eur', 'usd', 'gbp', 'chf']).default('eur'),
  orderId: z.string().cuid(),
  locationId: z.string().cuid().optional(),
  metadata: z.record(z.string()).optional(),
})

export const dynamic = 'force-dynamic'

export async function POST(req: Request) {
  try {
    const auth = await requireAuth(req, { permission: 'take_orders' })
    if (auth.error) return auth.error

    const { data, error } = await validateRequest(req, intentSchema, { maxBodySize: 32 * 1024 })
    if (error) return error

    // Preveri, da order pripada isti lokaciji
    const order = await db.order.findFirst({
      where: { id: data.orderId, locationId: data.locationId },
      select: { id: true, total: true, status: true },
    })
    if (!order) {
      return NextResponse.json({ error: 'Naročilo ni najdeno' }, { status: 404 })
    }

    // Ustvari Stripe PaymentIntent
    const gateway = new StripeGateway({
      secretKey: process.env.STRIPE_SECRET_KEY!,
      webhookSecret: process.env.STRIPE_WEBHOOK_SECRET!,
    })

    const result = await gateway.createPayment({
      amount: data.amount,
      currency: data.currency,
      orderId: data.orderId,
      description: `RestaurantOS Order ${data.orderId}`,
      metadata: {
        ...data.metadata,
        orderId: data.orderId,
        locationId: data.locationId || '',
      },
    })

    if (!result.success) {
      logger.error('STRIPE', `PaymentIntent failed: ${result.errorMessage}`)
      return NextResponse.json(
        { error: result.errorMessage, code: result.errorCode },
        { status: 400 }
      )
```

```
    }

    // Shrani v Payment tabelo
    await db.payment.create({
      data: {
        orderId: data.orderId,
        amount: data.amount / 100, // pretvori v EUR za DB
        method: 'card',
        gateway: 'stripe',
        gatewayTransactionId: result.gatewayTransactionId,
        status: 'pending',
      },
    })

    return NextResponse.json({
      clientSecret: result.clientSecret,
      paymentIntentId: result.gatewayTransactionId,
      amount: data.amount,
      currency: data.currency,
      status: result.status,
    })
  } catch (err) {
    return handleApiError(err, 'POST /api/payments/stripe-intent', 'Napaka pri ustvarjanju PaymentIntent')
  }
}
```

3.3 React komponenta: StripeCardInput

Komponenta za vnos kartičnih podatkov z uporabo @stripe/react-stripe-js. Uporablja Stripe Elements (PCI-compliant - kartični podatki nikoli ne pridejo do našega serverja).

```
// src/components/pos/StripeCardInput.tsx
'use client'
import { useState } from 'react'
import { loadStripe } from '@stripe/stripe-js'
import { Elements, CardElement, useStripe, useElements } from '@stripe/react-stripe-js'
import { Loader2, CreditCard, CheckCircle, AlertCircle } from 'lucide-react'

const stripePromise = loadStripe(process.env.NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY!)

interface StripeCardInputProps {
  amount: number  // v centih
  currency: string
  orderId: string
  locationId?: string
  onSuccess: (paymentIntentId: string) => void
  onError: (error: string) => void
}

const cardElementOptions = {
  style: {
    base: {
      fontSize: '16px',
      color: '#1d1c1a',
      '::placeholder': { color: '#8a8881' },
    },
    invalid: { color: '#9b4a43' },
  },
  hidePostalCode: true,
}

function PaymentForm({ amount, currency, orderId, locationId, onSuccess, onError }: StripeCardInputProps) {
  const stripe = useStripe()
  const elements = useElements()
```

```
const [processing, setProcessing] = useState(false)
const [error, setError] = useState<string | null>(null)

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault()
  if (!stripe || !elements) return

  setProcessing(true)
  setError(null)

  try {
    // 1. Ustvari PaymentIntent
    const intentRes = await fetch('/api/payments/stripe-intent', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ amount, currency, orderId, locationId }),
    })
    if (!intentRes.ok) throw new Error('Napaka pri ustvarjanju plačila')
    const { clientSecret, paymentIntentId } = await intentRes.json()

    // 2. Potrdi plačilo s Stripe
    const { error: stripeError, paymentIntent } = await stripe.confirmCardPayment(
      clientSecret,
      { payment_method: { card: elements.getElement(CardElement)! } }
    )

    if (stripeError) throw new Error(stripeError.message)
    if (paymentIntent.status === 'succeeded') {
      onSuccess(paymentIntentId)
    } else {
      throw new Error(`Status: ${paymentIntent.status}`)
    }
  } catch (err) {
    const msg = err instanceof Error ? err.message : 'Neznana napaka'
    setError(msg)
    onError(msg)
  } finally {
    setProcessing(false)
  }
}

return (
  <form onSubmit={handleSubmit} className="space-y-4">
    <div className="border rounded-lg p-4">
      <CardElement options={cardElementOptions} />
    </div>

    {error && (
      <div className="bg-red-50 border border-red-200 rounded p-3 flex items-center gap-2">
        <AlertCircle className="h-5 w-5 text-red-500" />
        <span className="text-sm text-red-700">{error}</span>
      </div>
    )}

    <button
      type="submit"
      disabled={!stripe || processing}
      className="w-full bg-[#86702b] text-white py-3 rounded-lg font-medium disabled:opacity-50"
    >
      {processing ? (
        <Loader2 className="mx-auto h-5 w-5 animate-spin" />
      ) : (
        `Plačaj ${(amount / 100).toFixed(2)} ${currency.toUpperCase()}`
      )}
    </button>
```

```
    </form>
  )
}

export function StripeCardInput(props: StripeCardInputProps) {
  return (
    <Elements stripe={stripePromise}>
      <PaymentForm {...props} />
    </Elements>
  )
}
```

3.4 Webhook handler (POSODOBITEV obstoječega)

Webhook prejema asinhrona obvestila od Stripe (npr. payment_intent.succeeded). RestaurantOS že ima webhook infrastrukturo (src/lib/payment-gateways/providers/stripe.ts) , a jo je treba registrirati pri Stripe v produkciji.

```
// src/app/api/payment-gateways/webhook/route.ts (POSODOBI)
import { NextResponse } from 'next/server'
import { StripeGateway } from '@/lib/payment-gateways/providers/stripe'
import { db } from '@/lib/db'
import { logger } from '@/lib/logger'

export const dynamic = 'force-dynamic'

export async function POST(req: Request) {
  const sig = req.headers.get('stripe-signature')
  if (!sig) return NextResponse.json({ error: 'Missing signature' }, { status: 400 })

  const body = await req.text()
  const gateway = new StripeGateway({
    secretKey: process.env.STRIPE_SECRET_KEY!,
    webhookSecret: process.env.STRIPE_WEBHOOK_SECRET!,
  })

  const event = gateway.verifyWebhook(body, sig)
  if (!event) {
    logger.warn('STRIPE', 'Invalid webhook signature')
    return NextResponse.json({ error: 'Invalid signature' }, { status: 400 })
  }

  switch (event.type) {
    case 'payment_intent.succeeded':
      await db.payment.updateMany({
        where: { gatewayTransactionId: event.data.object.id },
        data: { status: 'completed' },
      })
      logger.info('STRIPE', `Payment succeeded: ${event.data.object.id}`)
      break

    case 'payment_intent.payment_failed':
      await db.payment.updateMany({
        where: { gatewayTransactionId: event.data.object.id },
        data: { status: 'failed' },
      })
      logger.warn('STRIPE', `Payment failed: ${event.data.object.id}`)
      break

    case 'charge.refunded':
      await db.payment.updateMany({
        where: { gatewayTransactionId: event.data.object.payment_intent },
        data: { status: 'refunded' },
```

```
      })
      break
  }

  return NextResponse.json({ received: true })
}
```

3.5 Test kartice

Kartica	Scenarij	Pričakovan rezultat
4242 4242 4242 4242	Uspešno plačilo	succeeded
4000 0027 6000 3184	3-D Secure (SCA) required	requires_action
4000 0000 0000 9995	Ničelno stanje	insufficient_funds
4000 0000 0000 0069	Expired card	expired_card
4000 0000 0000 0119	Security code invalid	incorrect_cvc
4000 0082 6000 0179	Fraudulent (block)	card_declined

Tabela 3.2: Stripe test kartice za testiranje

3.6 Acceptance Criteria za P0-2

#	Kriterij	Test scenarij	Status
AC-1	PaymentIntent se ustvari	POST /stripe-intent → 200	☐
AC-2	StripeCardInput rendera	Mount komponento → prikaže CardElement	☐
AC-3	Uspešno plačilo z 4242	Test kartica → onSuccess klican	☐
AC-4	Padec plačila prikazan	9995 kartica → error prikazan	☐
AC-5	Webhook prejme event	Stripe dashboard test → DB update	☐
AC-6	Payment.status = completed	Po webhook → preveri DB	☐
AC-7	Refund deluje	POST refund → status refunded	☐
AC-8	Health check deluje	GET /payment-gateways?health=1	☐
AC-9	PCI compliant (no card data)	Network tab → kartica ne gre na naš server	☐
AC-10	Produkcijska Stripe povezava	Live keys → testno plačilo 0.50 EUR	☐

Tabela 3.3: Acceptance criteria za P0-2 Stripe

4. P0-3: PWA aplikacija

PWA (Progressive Web App) omogoča natakarjem uporabo RestaurantOS na tablicah brez interneta. Service Worker (v9) in manifest.json že delujeta. Ta specifikacija opisuje push notifications, install prompt UI in app icon set.

4.1 Obstoječi moduli (production-ready)

Modul	Datoteka	Status	Opis
Service Worker	public/sw.js	Production	681 vrstic, v9, cache strategije
Manifest	public/manifest.json	Production	8 ikon, standalone display
Offline Orders	src/lib/offline-orders/index.ts	Production	IndexedDB queue
Offline FURS	src/lib/offline-furs/index.ts	Production	FURS queue z 48h dovoljenim zamikom
Background Sync	public/sw.js	Production	Auto sinhronizacija
Offline fallback	public/offline.html	Production	Cache fallback stran

Tabela 4.1: Obstoječi PWA moduli

4.2 VAPID ključi generacija

VAPID (Voluntary Application Server Identification) ključi so potrebni za Web Push API. Generirajo se enkrat in se shranijo v .env.

```
# Generiraj VAPID ključe (enkrat, v Node.js REPL)
node -e "
const webpush = require('web-push');
const vapidKeys = webpush.generateVAPIDKeys();
console.log('NEXT_PUBLIC_VAPID_PUBLIC_KEY=' + vapidKeys.publicKey);
console.log('VAPID_PRIVATE_KEY=' + vapidKeys.privateKey);
"

# Dodaj v .env:
NEXT_PUBLIC_VAPID_PUBLIC_KEY=<public-key>
VAPID_PRIVATE_KEY=<private-key>
VAPID_SUBJECT=mailto:info@restaurantos.app
```

4.3 Nov API: POST /api/push/subscribe

Endpoint za subscripcijo natakarja na push notifications. Shranjuje PushSubscription v DB.

```
// src/lib/validations/push.ts
import { z } from 'zod'

export const subscribeSchema = z.object({
  endpoint: z.string().url(),
  keys: z.object({
    p256dh: z.string(),
    auth: z.string(),
  }),
```

```
  expirationTime: z.number().nullable().optional(),
})

// Prisma model (dodaj v schema.prisma):
// model PushSubscription {
//   id          String   @id @default(cuid())
//   employeeId  String
//   endpoint    String
//   keysP256dh  String
//   keysAuth    String
//   expirationTime Int?
//   createdAt   DateTime @default(now())
//   employee    Employee @relation(fields: [employeeId], references: [id])
//   @@unique([employeeId, endpoint])
// }

// src/app/api/push/subscribe/route.ts
import { NextResponse } from 'next/server'
import { requireAuth } from '@/lib/auth-middleware'
import { validateRequest, handleApiError } from '@/lib/api-utils'
import { subscribeSchema } from '@/lib/validations/push'
import { db } from '@/lib/db'

export const dynamic = 'force-dynamic'

export async function POST(req: Request) {
  try {
    const auth = await requireAuth(req, { permission: 'take_orders' })
    if (auth.error) return auth.error

    const { data, error } = await validateRequest(req, subscribeSchema)
    if (error) return error

    const employeeId = auth.session!.employeeId

    // Upsert subscription (idempotent)
    await db.pushSubscription.upsert({
      where: {
        employeeId_endpoint: {
          employeeId,
          endpoint: data.endpoint,
        },
      },
      update: {
        keysP256dh: data.keys.p256dh,
        keysAuth: data.keys.auth,
        expirationTime: data.expirationTime || null,
      },
      create: {
        employeeId,
        endpoint: data.endpoint,
        keysP256dh: data.keys.p256dh,
        keysAuth: data.keys.auth,
        expirationTime: data.expirationTime || null,
      },
    })

    return NextResponse.json({ success: true })
  } catch (err) {
    return handleApiError(err, 'POST /api/push/subscribe', 'Napaka pri subscripciji')
  }
}
```

4.4 Nov API: POST /api/push/send

Endpoint za pošiljanje push notification (samo admin/manager). Uporablja web-push knjižnico.

```
// src/app/api/push/send/route.ts
import { NextResponse } from 'next/server'
import webpush from 'web-push'
import { requireAuth } from '@/lib/auth-middleware'
import { handleApiError } from '@/lib/api-utils'
import { db } from '@/lib/db'
import { logger } from '@/lib/logger'

// Konfiguriraj web-push enkrat
webpush.setVapidDetails(
  process.env.VAPID_SUBJECT || 'mailto:info@restaurantos.app',
  process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY!,
  process.env.VAPID_PRIVATE_KEY!
)

export const dynamic = 'force-dynamic'

export async function POST(req: Request) {
  try {
    const auth = await requireAuth(req, { permission: 'manage_staff' })
    if (auth.error) return auth.error

    const { employeeId, title, body, data } = await req.json()

    const subscription = await db.pushSubscription.findMany({
      where: { employeeId },
    })

    if (subscription.length === 0) {
      return NextResponse.json({ error: 'Ni aktivnih subscriptionov' }, { status: 404 })
    }

    const payload = JSON.stringify({
      title,
      body,
      data: data || {},
      icon: '/icons/icon-192.png',
      badge: '/icons/badge-72.png',
      tag: 'restaurantos-notification',
      requireInteraction: false,
    })

    const results = await Promise.allSettled(
      subscription.map(sub =>
        webpush.sendNotification(
          {
            endpoint: sub.endpoint,
            keys: { p256dh: sub.keysP256dh, auth: sub.keysAuth },
          },
          payload
        )
      )
    )

    const succeeded = results.filter(r => r.status === 'fulfilled').length
    const failed = results.filter(r => r.status === 'rejected').length

    logger.info('PUSH', `Sent to ${succeeded}/${subscription.length} (failed: ${failed})`)

    return NextResponse.json({ succeeded, failed, total: subscription.length })
  } catch (err) {
```

```
    return handleApiError(err, 'POST /api/push/send', 'Napaka pri pošiljanju push')
  }
}
```

4.5 Service Worker posodobitev (push event)

Service Worker (public/sw.js) mora dobiti nov event listener za push events. Verzija cache se dvigne na v10.

```
// DODAJ v public/sw.js (na koncu datoteke)

// ============================================
// PUSH EVENT — Sprejmi push notification
// ============================================
self.addEventListener('push', (event) => {
  if (!event.data) return

  try {
    const data = event.data.json()
    const options = {
      body: data.body,
      icon: data.icon || '/icons/icon-192.png',
      badge: data.badge || '/icons/badge-72.png',
      tag: data.tag || 'restaurantos-notification',
      requireInteraction: data.requireInteraction || false,
      data: data.data || {},
      actions: data.actions || [],
      vibrate: [200, 100, 200],
    }

    event.waitUntil(
      self.registration.showNotification(data.title, options)
    )
  } catch (err) {
    console.error('[SW] Push napaka:', err)
  }
})

// ============================================
// NOTIFICATION CLICK — Odpre aplikacijo
// ============================================
self.addEventListener('notificationclick', (event) => {
  event.notification.close()

  const targetUrl = event.notification.data.url || '/'

  event.waitUntil(
    clients.matchAll({ type: 'window', includeUncontrolled: true }).then((clientList) => {
      for (const client of clientList) {
        if (client.url.includes(targetUrl) && 'focus' in client) {
          return client.focus()
        }
      }
      if (clients.openWindow) {
        return clients.openWindow(targetUrl)
      }
    })
  )
})
```

4.6 React komponenta: PushSubscriptionManager

```
// src/components/pwa/PushSubscriptionManager.tsx
```

```
'use client'
import { useEffect, useState } from 'react'
import { Bell, BellOff, Loader2 } from 'lucide-react'

export function PushSubscriptionManager() {
  const [supported, setSupported] = useState(false)
  const [subscribed, setSubscribed] = useState(false)
  const [permission, setPermission] = useState<NotificationPermission>('default')
  const [loading, setLoading] = useState(false)

  useEffect(() => {
    setSupported('serviceWorker' in navigator && 'PushManager' in window)
    setPermission(Notification.permission)

    // Preveri obstoječo subscripcijo
    if ('serviceWorker' in navigator) {
      navigator.serviceWorker.ready.then(async (reg) => {
        const sub = await reg.pushManager.getSubscription()
        setSubscribed(!!sub)
      })
    }
  }, [])

  const subscribe = async () => {
    setLoading(true)
    try {
      // 1. Prosi za dovoljenje
      const perm = await Notification.requestPermission()
      setPermission(perm)
      if (perm !== 'granted') return

      // 2. Registriraj Service Worker (če še ni)
      const reg = await navigator.serviceWorker.ready

      // 3. Ustvari PushSubscription
      const sub = await reg.pushManager.subscribe({
        userVisibleOnly: true,
        applicationServerKey: process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY,
      })

      // 4. Pošlji na server
      const res = await fetch('/api/push/subscribe', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(sub),
      })

      if (res.ok) setSubscribed(true)
    } catch (err) {
      console.error('Push subscribe failed:', err)
    } finally {
      setLoading(false)
    }
  }

  if (!supported) {
    return (
      <div className="text-sm text-gray-500 flex items-center gap-2">
        <BellOff className="h-4 w-4" /> Push notifications niso podprte
      </div>
    )
  }

  return (
    <button
```

```
      onClick={subscribe}
      disabled={loading || subscribed || permission === 'denied'}
      className="flex items-center gap-2 px-4 py-2 bg-[#86702b] text-white rounded-lg disabled:opacity-50"
    >
      {loading ? <Loader2 className="h-4 w-4 animate-spin" /> : <Bell className="h-4 w-4" />}
      {subscribed ? 'Obvestila aktivirana' : 'Vklopi obvestila'}
    </button>
  )
}
```

4.7 Install prompt komponenta

```
// src/components/pwa/InstallPrompt.tsx
'use client'
import { useState, useEffect } from 'react'
import { Download, X } from 'lucide-react'

interface BeforeInstallPromptEvent extends Event {
  prompt: () => Promise<void>
  userChoice: Promise<{ outcome: 'accepted' | 'dismissed' }>
}

export function InstallPrompt() {
  const [deferredPrompt, setDeferredPrompt] = useState<BeforeInstallPromptEvent | null>(null)
  const [show, setShow] = useState(false)
  const [installed, setInstalled] = useState(false)

  useEffect(() => {
    // Preveri, ali je že instalirana
    if (window.matchMedia('(display-mode: standalone)').matches) {
      setInstalled(true)
      return
    }

    const handler = (e: Event) => {
      e.preventDefault()
      setDeferredPrompt(e as BeforeInstallPromptEvent)
      // Pokaži po 3 obiskih (localStorage counter)
      const visits = parseInt(localStorage.getItem('visits') || '0') + 1
      localStorage.setItem('visits', String(visits))
      if (visits >= 3) setShow(true)
    }

    window.addEventListener('beforeinstallprompt', handler)
    window.addEventListener('appinstalled', () => setInstalled(true))

    return () => {
      window.removeEventListener('beforeinstallprompt', handler)
    }
  }, [])

  const handleInstall = async () => {
    if (!deferredPrompt) return
    await deferredPrompt.prompt()
    const choice = await deferredPrompt.userChoice
    if (choice.outcome === 'accepted') {
      setInstalled(true)
      setShow(false)
    }
    setDeferredPrompt(null)
  }

  if (installed || !show) return null
```

```
  return (
    <div className="fixed bottom-4 right-4 bg-white border border-gray-200 rounded-lg shadow-lg p-4 max-w-sm z-50">
      <button
        onClick={() => setShow(false)}
        className="absolute top-2 right-2 text-gray-400 hover:text-gray-600"
      >
        <X className="h-4 w-4" />
      </button>
      <div className="flex items-start gap-3">
        <Download className="h-6 w-6 text-[#86702b] flex-shrink-0 mt-1" />
        <div>
          <h3 className="font-semibold text-sm">Namesti RestaurantOS</h3>
          <p className="text-xs text-gray-600 mt-1">
            Dodaj na domači zaslon za hitri dostop in offline delo.
          </p>
          <button
            onClick={handleInstall}
            className="mt-3 bg-[#86702b] text-white text-sm px-4 py-2 rounded"
          >
            Namesti
          </button>
        </div>
      </div>
    </div>
  )
}
```

4.8 Acceptance Criteria za P0-3

#	Kriterij	Test scenarij	Status
AC-1	VAPID ključi generirani	Preveri .env	☐
AC-2	PushSubscriptionManager rendera	Mount → prikaže gumb	☐
AC-3	Permission request deluje	Klik → browser prompt	☐
AC-4	Subscription shranjen v DB	Preveri PushSubscription tabelo	☐
AC-5	POST /api/push/send pošlje	Test pošiljanje → prejmi notifikacijo	☐
AC-6	SW push event deluje	Pošlji push → prikaže se notification	☐
AC-7	Notification click odpre app	Klik → focus/open app	☐
AC-8	InstallPrompt se prikaže po 3 obiskih	3x reload → prikaže se	☐
AC-9	App ikone prikazane pravilno	Add to home screen → ikona	☐
AC-10	SW verzija v10 aktivna	Preveri v Application tab	☐

Tabela 4.2: Acceptance criteria za P0-3 PWA

5. Environment spremenljivke

Vse nove env spremenljivke, ki jih potrebujemo za P0 implementacijo. Dodaj v .env in Vercel environment variables.

```
# === FURS (P0-1) ===
FURS_ENV="test"                          # ali "production"
FURS_CERT_PATH="./certs/furs-test.p12"  # path do certifikata
FURS_CERT_PASSWORD=""                    # geslo certifikata
FURS_TAX_NUMBER=""                       # davčna številka
FURS_ALLOW_SIMULATION="false"           # false = fail-closed

# === STRIPE (P0-2) ===
STRIPE_SECRET_KEY="sk_test_..."         # test key, zamenjaj za sk_live_...
STRIPE_PUBLISHABLE_KEY="pk_test_..."    # NEXT_PUBLIC_ prefix za client
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."
STRIPE_WEBHOOK_SECRET="whsec_..."       # iz Stripe dashboard

# === VAPID (P0-3) ===
NEXT_PUBLIC_VAPID_PUBLIC_KEY="<generated>"
VAPID_PRIVATE_KEY="<generated>"
VAPID_SUBJECT="mailto:info@restaurantos.app"

# === SENTRY (že obstoječe) ===
SENTRY_DSN="https://..."
NEXT_PUBLIC_SENTRY_DSN="https://..."
```

5.1 Vercel environment variables

Vse zgornje spremenljivke (razen NEXT_PUBLIC_*) morajo biti nastavljene v Vercel dashboard → Settings → Environment Variables. NEXT_PUBLIC_ spremenljivke so expose-ane clientu in morajo biti nastavljene za vse environmente (Production, Preview, Development).

6. Zaključek in naslednji koraki

Tehnična specifikacija podrobno opisuje implementacijo P0 prioritete. Skupni napor je 42 človek-dnevov (8 tednov z 1 FTE). Vsaka komponenta ima jasno definirane API contracts, TypeScript sheme, React komponente in acceptance criteria.

6.1 Ključne odgovornosti

Komponenta	Lead	Napor	Rok
P0-1 FURS cert upload UI	Backend developer	10d	Teden 1-2
P0-1 FURS test/produkcija	Backend + stranka	7d	Teden 2-3
P0-2 Stripe POS UI	Frontend developer	8d	Teden 1-2
P0-2 Stripe test/webhook	Backend developer	5d	Teden 3
P0-3 PWA push notifications	Fullstack developer	6d	Teden 4-5
P0-3 PWA install prompt	Frontend developer	3d	Teden 5
P0-3 PWA app icons	Designer	2d	Teden 6
E2E testi	QA / Backend	5d	Teden 6-7
Dokumentacija	Tech Lead	3d	Teden 7
Deploy + finalni review	Tech Lead	2d	Teden 8

Tabela 6.1: Odgovornosti po komponentah

6.2 Definition of Done (DoD)

- **Koda**: implementirana po specifikaciji, prestala E2E testi, brez kritičnih bugov
- **Testi**: acceptance criteria (AC-1 do AC-30) vsi preverjeni in označeni
- **Dokumentacija**: README posodobljen, ARCHITECTURE.md posodobljen, JSDoc komentarji
- **Code review**: minimalno 1 reviewer, odobreno pred merge
- **Deploy**: deployed v produkcijo, Sentry monitoring aktiven, brez napak v 24h
- **Komunikacija**: stranka obveščena o zaključku, navodila za uporabo poslana

6.3 Tveganja in mitigacije

Tveganje	Verjetnost	Vpliv	Mitigacija
Stranka ne pridobi FURS certifikata	Srednja	Visok	Začni postopek takoj, 2 tedna lead time
Stripe račun zavrnjen	Nizka	Visok	Uporabi SumUp kot backup (EU friendly)

Tveganje	Verjetnost	Vpliv	Mitigacija
FURS testno okolje nedosegljivo	Nizka	Srednji	Implementiraj retry z exponential backoff
Service Worker cache konflikti	Srednja	Nizki	Verzija v10, force update na activate
Push notifications ne delujejo na iOS	Visoka	Srednji	iOS 16.4+ podpira, dodaj fallback SMS
Stripe webhook signatura neveljavna	Nizka	Visok	Testiraj z Stripe CLI locally

Tabela 6.2: Tveganja in mitigacije

ZAKLJUČEK

Specifikacija je popolna. Razvojna ekipa ima vse potrebno za implementacijo. Priporočam takojšen začetek (1. oktober 2025) z dvema vzporednima tokovima: (1) FURS cert upload UI + testiranje, (2) Stripe POS UI + testiranje. PWA push notifications sledijo od tedna 4. Do 30. novembra 2025 bo RestaurantOS pripravljen za prodajo slovenskim restavracijam.